

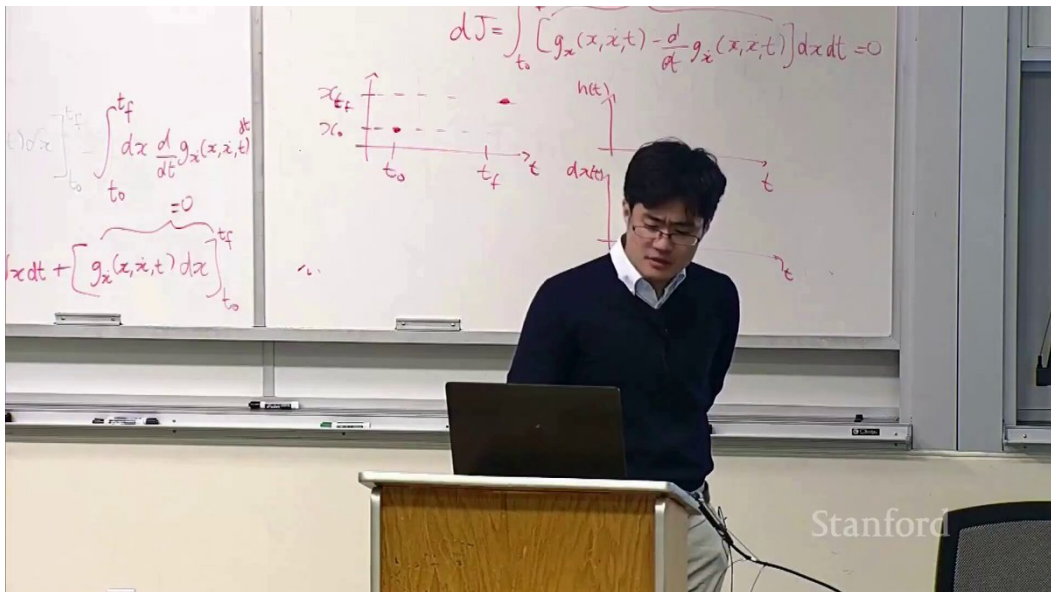
课程笔记

现代大语言模型架构与超参数

从共识到例外——Stanford CS336 2026 Lecture 3 高密度中文讲义

Tatsunori Hashimoto (讲授) / lecture-to-notes (整理)

2026 年 7 月 11 日



视频作者 / 频道: Stanford Online (原课程) / Vikas Gupta @vicky_grok (X 转发)

发布日期: 原课程视频: 2026-04-15; X 转发: 2026-07-10

视频时长: 01:29:03

X 视频链接: https://x.com/vicky_grok/status/2075594420163092606/video/1

课程主页: <https://cs336.stanford.edu/>

官方课件: Lecture 3: Architectures

目录

1 阅读指南与课程定位	2
1.1 如何使用这份讲义	2
1.2 本章小结	2
2 现代 Transformer 基线：哪些选择已接近共识	2
2.1 跨模型调查能告诉我们什么	3
2.2 一个实用的基线策略	4
2.3 本章小结	4
3 残差通路与归一化：让深网络可训练	4
3.1 Pre-Norm 与 Post-Norm 的根本差别	4
3.2 LayerNorm 与 RMSNorm	5
3.3 为什么 FLOPs 不能代表运行时间	6
3.4 本章小结	7
4 前馈网络、门控激活与并行块	7
4.1 从 ReLU / GELU 到 GLU 家族	7
4.2 串行块与并行块	8
4.3 本章小结	9
5 位置表示与 RoPE：把顺序写入注意力	9
5.1 RoPE 的核心目标：内积只感知相对距离	10
5.2 实现与外推时的常见陷阱	12
5.3 本章小结	12
6 尺寸超参数：宽度、深度、头数与词表	12
6.1 前馈层维度	13
6.2 注意力头：总维度比头数更关键	13
6.3 宽深比与扩展路径	14
6.4 词表大小是一项系统设计	15
6.5 本章小结	15
7 正则化与优化动力学：旧概念的新解释	16
7.1 Dropout 消退，权重衰减保留	16
7.2 权重衰减不只是“防过拟合”	16

7.3 本章小结	17
8 训练稳定性：控制 Softmax 的危险尺度	17
8.1 语言模型中有两处关键 Softmax	18
8.2 z-loss：约束输出归一化常数	18
8.3 QK-Norm：在注意力 Softmax 之前控幅	19
8.4 稳定性实验该怎样设计	20
8.5 本章小结	20
9 推理系统：Prefill、Decode 与 KV 缓存	20
9.1 Prefill：矩阵大、复用高、较偏计算受限	21
9.2 Decode：顺序依赖强、算术强度低	21
9.3 MQA：所有查询头共享一组 K/V	22
9.4 GQA：系统效率与表达能力的折中	23
9.5 本章小结	24
10 长上下文：稀疏、滑动窗口与混合注意力	24
10.1 从稀疏模式到滑动窗口	24
10.2 交错全局层与局部层	25
10.3 混合架构仍在快速演化	26
10.4 本章小结	27
11 总结与延伸	27
11.1 讲者总结：共识、例外与变化最快的边界	27
11.2 一张可执行的设计清单	27
11.3 进一步推导：从组件思维转向约束思维	28
11.4 建议练习	28
11.5 拓展阅读	29
11.6 本章小结	29

1 阅读指南与课程定位

这节课不是一份“照着抄就能得到最强模型”的配方，而是一张架构决策地图。讲者 Tatsunori Hashimoto 以当前主流稠密语言模型为样本，追问三个问题：哪些设计已经成为可靠默认值？哪些选择主要服务于 GPU 与推理系统？哪些新改动是在昂贵训练中为稳定性购买保险？

全课主线

架构选择同时受三类约束支配：**学习与泛化能力、训练 / 推理系统效率、数值稳定性**。只比较参数量或 FLOPs，往往无法预测真实吞吐、长上下文成本与训练失败风险。

本讲的经验来源有明确层级。最可靠的是亲手实现、训练并做受控消融；其次是跨模型调查表；最后才是从单个成功模型反推因果。表格能揭示“共识”，却不能自动证明某一系列就是性能来源：成功模型常常一次改变多个因素，而且训练数据、优化器、预算与硬件也不同。

材料对齐说明

本讲义以用户提供的 X 视频为时间轴。为修复 X 自动字幕后半段的明显错乱，制作时将其与 Stanford Online 发布的同一场 2026 年 Lecture 3 官方录像进行画面、音频和时长核验，并把官方人工字幕整体对齐到 X 时间轴；所有插图仍取自 X 视频文件。图注中的时间可直接用于回看原链接。

1.1 如何使用这份讲义

建议先看每节的“判断框架”和“小结”，再回到公式与图表。模型设计时，不要把“现代模型常用”误读为“在任何规模、硬件和数据上都最优”；应从稳定默认值出发，只在存在明确瓶颈和可验证收益时偏离。

1.2 本章小结

本课讨论的不是组件清单，而是约束下的资源配置。读表格时要区分相关性与因果；做设计时要同时记录质量、吞吐、显存、训练稳定性和实现复杂度。

2 现代 Transformer 基线：哪些选择已接近共识

原始 Transformer 奠定了自注意力、前馈网络、残差连接与归一化的基本骨架，但现代自回归语言模型通常使用一套更接近 LLaMA 的基线：Pre-Norm、RMSNorm、旋转位置编码（RoPE）、SwiGLU、无偏置线性层，以及面向因果语言建模的掩码注意力。这些变化并非都来自同一种动机：有的改善梯度传播，有的减少内存访问，有的提升参数利用率，有的服务长上下文。

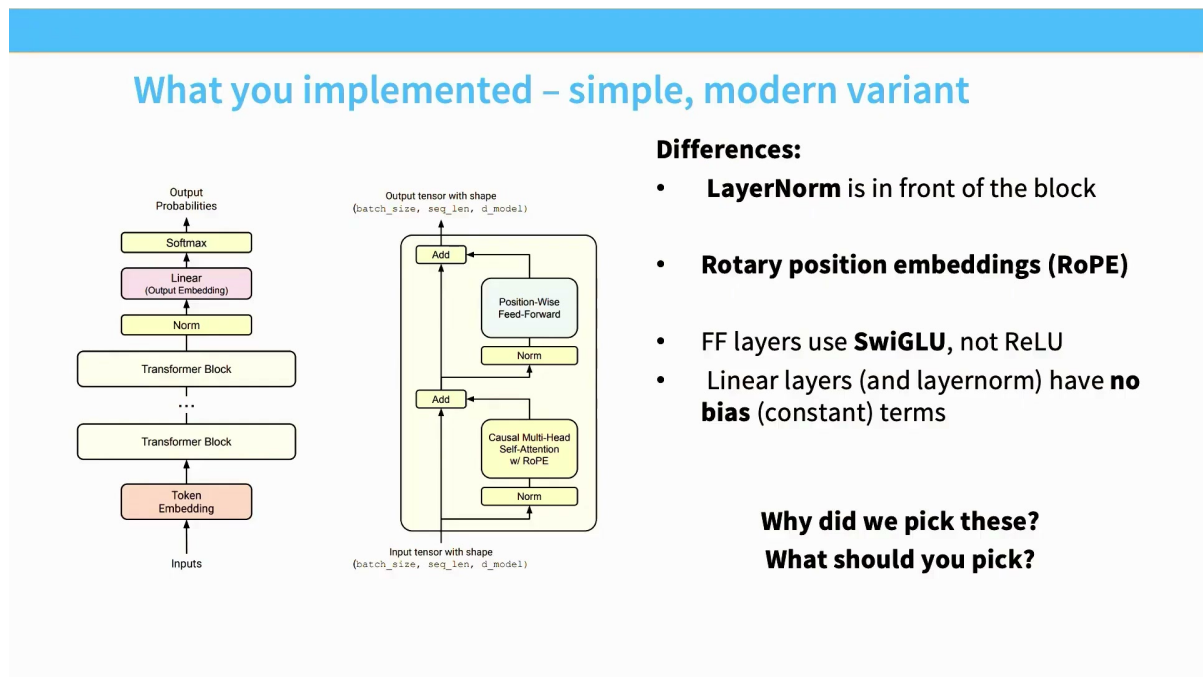


图 1: 课程作业采用的现代 Transformer 基线: Pre-Norm、RoPE、SwiGLU 与无偏置线性层。¹

2.1 跨模型调查能告诉我们什么

把 GPT 系列、PaLM、LLaMA、Gemma、Qwen 等模型放在同一张表里，可以快速识别高频组合：Pre-Norm 几乎一统主流稠密模型；RMSNorm 与 SwiGLU 在 LLaMA 之后十分常见；RoPE 成为位置处理的强默认值；偏置项和 dropout 则常被移除。然而，表格中的例外同样重要：它们说明设计空间仍未收敛为唯一答案。

¹视频画面时间区间：00:02:15–00:02:30。

Let's look at the data (on dense architectures)

Learn from the many other models (and papers) out there

We will talk through many major architecture and hyperparameter variants.

- What do all these models have in common?
- What parts vary?
- What can we learn from this?

图 2: 主流自回归语言模型的架构调查表: 共识清晰, 但每一列都仍存在成功例外。²

调查表不是因果实验

“所有强模型都用了某组件”只能说明它是可行且流行的工程选择。要判断组件是否必要, 需要在相同数据、规模、优化器与训练预算下做消融; 否则收益可能来自同时变化的训练配方。

2.2 一个实用的基线策略

新项目宜先复现一套充分验证的现代基线, 建立可重复的损失、吞吐与稳定性曲线。之后每次只改变一个关键因素, 并至少记录: 验证损失、每 token 计算量、tokens/s、峰值显存、通信量、训练尖峰和恢复行为。这样, 架构讨论才从“模型品牌比较”转化为可复核的设计证据。

2.3 本章小结

现代基线已形成, 但尚不存在唯一架构。最稳妥的工作方式是: 先站在成熟默认值上, 再用受控实验解释每一次偏离; 既观察平均性能, 也观察尾部失败和系统代价。

3 残差通路与归一化: 让深网络可训练

3.1 Pre-Norm 与 Post-Norm 的根本差别

残差流可以理解为贯穿全部层的信息高速公路。Post-Norm 在残差相加之后做归一化; Pre-Norm 则先归一化输入, 再把子层产生的增量写回未经归一化的残差流。对第 ℓ 层, 二者可概括为:

²视频画面时间区间: 00:04:30–00:04:45。

$$\text{Post-Norm: } x_{\ell+1} = \text{Norm}(x_{\ell} + F_{\ell}(x_{\ell})),$$

$$\text{Pre-Norm: } x_{\ell+1} = x_{\ell} + F_{\ell}(\text{Norm}(x_{\ell})).$$

- x_{ℓ} : 第 ℓ 层进入残差通路的表示;
- F_{ℓ} : 注意力或前馈子层;
- Norm: LayerNorm、RMSNorm 或其变体。

Pre-Norm 的关键不是“多了一个归一化”，而是保留了从浅层到深层的恒等路径。梯度可以沿残差通路更直接地传播，因此深层网络通常更容易优化。代价是各层对残差流持续累加更新，最终常需额外的输出归一化。

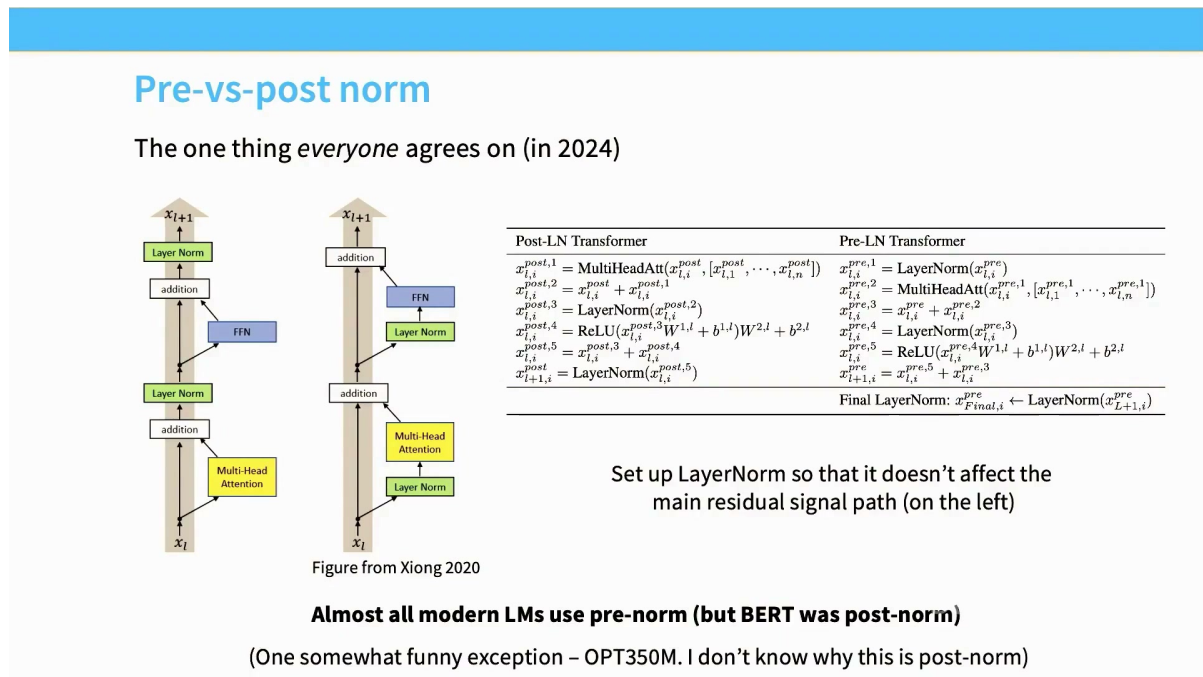


图 3: Post-Norm 与 Pre-Norm: 归一化位置决定了残差高速公路是否保持“干净”。³

判断要点

Pre-Norm 的主要价值是优化稳定性，而非直接增加表达能力。近年来也出现“无残差 Post-Norm”“双重归一化”等成功方案，但这些应视为重新设计信号与梯度尺度的完整系统，而不是随意挪动一层 Norm。

3.2 LayerNorm 与 RMSNorm

LayerNorm 同时去除均值并按标准差缩放；RMSNorm 不做均值中心化，只按均方根缩放。忽略可学习偏置时，它们可写为：

³ 视频画面时间区间：00:08:00–00:08:15。

$$\begin{aligned}\text{LN}(x)_i &= \gamma_i \frac{x_i - \mu}{\sqrt{\sigma^2 + \varepsilon}}, \\ \text{RMSNorm}(x)_i &= \gamma_i \frac{x_i}{\sqrt{\frac{1}{d} \sum_{j=1}^d x_j^2 + \varepsilon}}.\end{aligned}$$

- x_i : 一个 token 隐状态的第 i 个通道; d : 隐藏维度;
- μ, σ^2 : 同一 token 跨通道的均值与方差;
- γ_i : 可学习缩放; ε : 防止除零的稳定常数。

RMSNorm 省去了均值计算与减法，结构更简单，也更便于高效内核融合。它不是“近似错误的 LayerNorm”，而是只保留尺度不变性、放弃平移不变性的另一种归一化假设。大量现代语言模型表明，这个假设通常足够。

LayerNorm vs RMSNorm

Original transformer: **LayerNorm** – normalizes the mean and variance across d_{model}

$$y = \frac{x - \mathbb{E}[x]}{\sqrt{\text{Var}[x] + \epsilon}} * \gamma + \beta$$

Many modern LMs: **RMSNorm** – does not subtract mean or add a bias term

$$y = \frac{x}{\sqrt{\|x\|_2^2 + \varepsilon}} * \gamma$$

Notable models:

GPT3/2/1, OPT, GPT-J, BLOOM

Notable models:

LLaMA-family, PaLM, Chinchilla, T5

图 4: LayerNorm 与 RMSNorm 的运算差异, 以及主流模型中的采用情况。⁴

3.3 为什么 FLOPs 不能代表运行时间

归一化操作的 FLOPs 占比很小，却可能消耗显著运行时间。原因是它通常受显存带宽、kernel launch、同步和中间张量读写支配。讲者给出的例子中，统计归一化约占 0.17% 的 FLOPs，却可能在某些负载中占到约四分之一的运行时间。

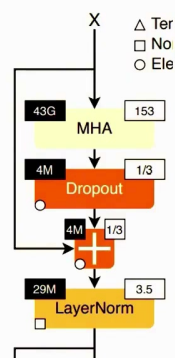
⁴视频画面时间区间：00:14:00—00:14:15。

Why RMSNorm (2)

Important lesson: FLOPS are not runtime! (we will discuss this in far more detail later)

Operator class	% flop	% Runtime
△ Tensor contraction	99.80	61.0
□ Stat. normalization	0.17	25.5
○ Element-wise	0.03	13.5

RMSNorm can still matter due to the importance of *data movement*



Left top ("43G") is FLOPS
Right top ("153") is the FLOP-to-memory ratio

[Ivanov et al 2023]

图 5: FLOPs 与墙钟时间并不等价: 低算术强度算子可能被数据搬运主导。⁵

系统侧检查清单

比较归一化方案时, 应在目标硬件和真实序列长度上测量融合后的端到端吞吐; 同时检查是否多次读写残差流、是否能与相邻线性层融合、是否引入额外同步。只看理论 FLOPs 会系统性低估逐元素算子的代价。

3.4 本章小结

Pre-Norm 通过保留恒等残差路径改善深层优化; RMSNorm 用更少操作实现有效的尺度控制。两者的流行同时体现了数值与系统动机, 而真实效率必须用端到端运行时间验证。

4 前馈网络、门控激活与并行块

4.1 从 ReLU / GELU 到 GLU 家族

Transformer 的前馈网络对每个 token 独立执行通道混合。经典形式先升维、施加非线性、再降维。GLU 家族把升维结果拆成“内容分支”和“门分支”, 通过逐元素乘法让一个分支调节另一个分支:

$$\text{SwiGLU}(x) = [\text{SiLU}(xW_g) \odot (xW_u)] W_d.$$

- W_g 、 W_u : 门分支与内容分支的上投影矩阵;
- W_d : 将中间表示投回模型维度的下投影矩阵;

⁵视频画面时间区间: 00:16:00–00:16:15。

- $\odot$: 逐元素乘法; $\text{SiLU}(z) = z \sigma(z)$ 。

GeGLU 用 GELU 作为门激活, SwiGLU 用 SiLU / Swish。Google 系模型常见 GeGLU, LLaMA 及其后继常见 SwiGLU。门控通常能带来稳定的小幅收益, 但 GPT-3 等非 GLU 模型的成功提醒我们: 它是高质量默认值, 不是语言建模成立的必要条件。

Gated variants of standard FF layers	
GeGLU $\text{FFN}_{\text{GeGLU}}(x, W, V, W_2) = (\text{GELU}(xW) \otimes xV)W_2$	Notable models: T5 v1.1, mT5, LaMDA, Phi3, Gemma 2, Gemma 3, Gemma 4
SwiGLU (swish is $x * \text{sigmoid}(x)$) $\text{FFN}_{\text{SwiGLU}}(x, W, V, W_2) = (\text{Swish}_1(xW) \otimes xV)W_2$	Notable models: LLaMa 1/2/3, PaLM, Mistral, OLMo, <i>most models post 2023</i>
Note: Gated models use smaller dimensions for the d_{ff} by 2/3	

图 6: GeGLU 与 SwiGLU: 门控分支使用不同的平滑激活函数。⁶

为什么 GLU 的中间维度常小于 $4d$

门控 FFN 有两套上投影, 若仍令每套中间维度等于 $4d$, 参数与计算都会显著增加。实践中常把门控中间维度设为约 $\frac{8}{3}d$, 使三块大矩阵的总参数量接近传统 $4d$ FFN, 从而进行近似等预算比较。

4.2 串行块与并行块

标准 Transformer 块先做注意力, 再把结果交给 FFN; 并行块则让注意力和 FFN 读取同一归一化输入, 并把两者输出一次性加回残差流:

$$\begin{aligned}
 x' &= x + \text{Attn}(\text{Norm}(x)), \\
 x_{\text{serial}}^+ &= x' + \text{FFN}(\text{Norm}(x')), \\
 x_{\text{parallel}}^+ &= x + \text{Attn}(\text{Norm}(x)) + \text{FFN}(\text{Norm}(x)).
 \end{aligned}$$

- x' : 串行结构中注意力写回后的中间残差;
- x_{serial}^+ : 串行块输出, FFN 可以读取本层注意力结果;
- x_{parallel}^+ : 并行块输出, 两条分支之间没有本层内依赖。

⁶视频画面时间区间: 00:23:45–00:24:00。

并行块可以共享一次归一化，并把多个矩阵乘法调度得更紧凑，理论上更利于并行与融合；但它改变了层内信息依赖。大多数近期模型仍采用串行块，说明系统收益需要与表达能力、现有内核和训练经验一起评估。

Parallel layers

A few models (GPT-J, PaLM, GPT-NeoX) do parallel layers. Originally in GPT-J

Parallel Layers – We use a “parallel” formulation in each Transformer block (Wang & Komatsuzaki, 2021), rather than the standard “serialized” formulation. Specifically, the standard formulation can be written as:

$$y = x + \text{MLP}(\text{LayerNorm}(x + \text{Attention}(\text{LayerNorm}(x))))$$

Whereas the parallel formulation can be written as:

$$y = x + \text{MLP}(\text{LayerNorm}(x)) + \text{Attention}(\text{LayerNorm}(x))$$

The parallel formulation results in roughly 15% faster training speed at large scales, since the MLP and Attention input matrix multiplications can be fused. Ablation experiments showed a small quality degradation at 8B scale but no quality degradation at 62B scale, so we extrapolated that the effect of parallel layers should be quality neutral at the 540B scale.

If implemented right, LayerNorm can be shared, and matrix multiplies can be fused

Recent Models: Cohere Command A, Falcon 2 11B, Command R+

图 7: 串行与并行 Transformer 块：并行形式减少依赖链，但改变了层内计算图。⁷

4.3 本章小结

SwiGLU 是参数利用率较高的现代默认选择，但要按等预算调整中间维度。并行块展示了“架构即调度”的一面：减少归一化和依赖可能提升系统效率，却并不自动保证质量收益。

5 位置表示与 RoPE：把顺序写入注意力

自注意力本身对输入排列不敏感；如果不给位置信息，模型无法区分“猫追狗”和“狗追猫”的顺序。位置处理大致经历了四类方案：固定正弦编码、可学习绝对位置向量、相对位置偏置，以及把位置变换直接施加到查询和键上的 RoPE。

⁷视频画面时间区间：00:28:15–00:28:30。

Many variations in position embeddings

Sine embeddings: add sines and cosines that enable localization

$$\text{Embed}(x, i) = v_x + PE_{pos}$$

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

Notable models:

Original transformer

Absolute embeddings: add a position vector to the embedding

$$\text{Embed}(x, i) = v_x + u_i$$

Notable models:

GPT1/2/3, OPT

Relative embeddings: add a vector to the *attention computation*

$$e_{ij} = \frac{x_i W^Q (x_j W^K + a_{ij}^K)^T}{\sqrt{d_z}}$$

Notable models:

T5, Gopher, Chinchilla

Rope embeddings (next slides..)

Notable models:

GPTJ, PaLM, LLaMA
Most 2024+ models

图 8: 位置表示家族: 从绝对向量、相对偏置到作用在查询与键上的旋转变换。⁸

5.1 RoPE 的核心目标: 内积只感知相对距离

注意力分数来自查询与键的内积。理想的位置变换 f 应让两个位置变换后的内积只依赖内容和相对位移, 而不是绝对坐标:

$$\langle f(q, i), f(k, j) \rangle = g(q, k, j - i).$$

- q, k : 某个注意力头中的查询和键向量;
- i, j : 对应 token 的绝对位置;
- g : 只通过 $j - i$ 感知相对距离的打分函数。

RoPE 将通道两两配对, 在每个二维子空间内按位置旋转。直观上, 每个 token 的向量像钟表指针一样随位置转动; 两个指针的夹角天然等于两次旋转角之差, 因此点积能编码相对位置。

⁸视频画面时间区间: 00:31:45–00:32:00。

RoPE: rotary position embeddings

How can we solve this problem?

- We want our embeddings to be invariant to absolute position
- We know that inner products are invariant to arbitrary rotation.

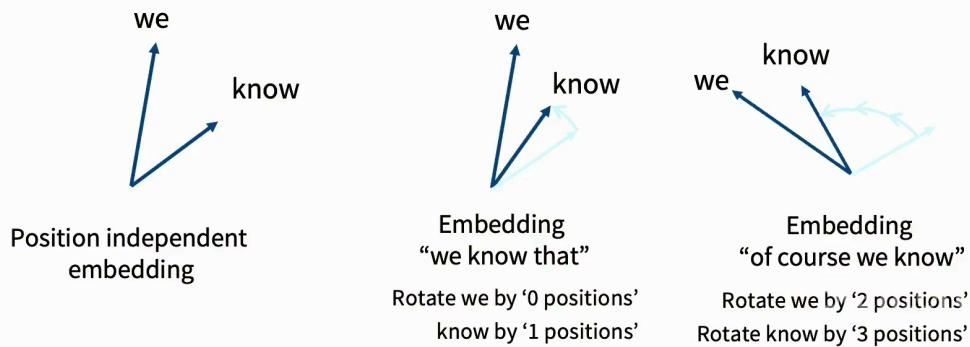


图 9: RoPE 的二维直觉: 对内容向量按位置旋转, 内积由相对旋转角决定。⁹

对第 m 个二维通道对, 可定义旋转矩阵:

$$R_m(i) = \begin{bmatrix} \cos(i\theta_m) & -\sin(i\theta_m) \\ \sin(i\theta_m) & \cos(i\theta_m) \end{bmatrix}, \quad (R_m(i)q)^\top R_m(j)k = q^\top R_m(j-i)k.$$

- θ_m : 第 m 个通道对的旋转频率, 通常按几何级数分布;
- $R_m(i)$: 位置 i 对该二维子空间施加的正交旋转;
- 等式使用 $R_m(i)^\top R_m(j) = R_m(j-i)$, 因此绝对位置相消。

⁹视频画面时间区间: 00:34:45-00:35:00。

The actual RoPE math

Multiply with sines and cosines

$$f_{\{q,k\}}(\mathbf{x}_m, m) = \mathbf{R}_{\Theta, m}^d \mathbf{W}_{\{q,k\}} \mathbf{x}_m \quad (14)$$

$$\mathbf{R}_{\Theta, m}^d = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix} \quad (15)$$

Difference with sine embeddings – not additive, no cross terms

图 10: RoPE 的分块旋转矩阵：不同通道对使用不同频率，覆盖多尺度位置变化。¹⁰

5.2 实现与外推时的常见陷阱

RoPE 应施加在每次注意力计算的 Q 、 K 上，而不是把旋转永久写进 token embedding。缓存解码时，缓存中的键必须已经带有其原始位置旋转，新查询则使用当前时间步的位置。频率基数、缩放方法与训练长度共同决定长度外推行为；仅把推理上下文窗口调大，不能保证模型在新距离上仍能可靠利用信息。

RoPE 不是免费的无限上下文

相对位置结构缓解了绝对位置表的长度限制，但模型仍只在有限距离分布上训练。长上下文能力还取决于训练数据、频率缩放、注意力模式、KV 缓存预算和检索行为；必须用真实长序列任务验证。

5.3 本章小结

RoPE 把绝对位置旋转转化为注意力内积中的相对位移，是现代模型的强默认值。其优雅代数结构不等于无限外推；实现中需正确处理 Q/K 旋转、缓存位置和频率缩放。

6 尺寸超参数：宽度、深度、头数与词表

架构表中最醒目的往往是组件名称，但训练成本与容量分配更直接地由尺寸超参数决定。讲者强调，许多比例在成功模型中聚集到窄区间；这些比例是很好的起点，却不是物理定律。

¹⁰ 视频画面时间区间：00:37:45–00:38:00。

6.1 前馈层维度

非门控 FFN 的经典经验值是 $d_{\text{ff}} \approx 4d_{\text{model}}$ 。对含两条上投影的 GLU，为保持近似等参数与等计算，常用：

$$d_{\text{ff, GLU}} \approx \frac{8}{3}d_{\text{model}}.$$

- d_{model} ：残差流 / 隐藏状态宽度；
- d_{ff} ：FFN 的中间通道数；
- 系数 8/3：补偿 GLU 的两条上投影，使总矩阵参数接近传统 $4d$ FFN。

工程实现还会把 d_{ff} 向上取整到适合 Tensor Core、张量并行或量化分组的倍数，因此论文中的比例常有小幅偏离。

Surprising (?) consensus hyperparameter 1

Feedforward – model dimension ratio.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

There are two dimensions that are relevant – the feedforward dim (d_{ff}) and model dim (d_{model}). What should their relationship be?

$$\mathbf{d_{ff} = 4 d_{model}}$$

This is *almost always* true. There's just a few exceptions.

图 11：传统 FFN 的经验比例：中间维度通常约为模型维度的四倍。¹¹

6.2 注意力头：总维度比头数更关键

多头注意力通常满足 $n_{\text{heads}}d_{\text{head}} \approx d_{\text{model}}$ 。增加头数并不会凭空增加总容量，而是把查询空间切成更多子空间；固定模型宽度时，头数与每头维度互相制约。每头维度也受高效矩阵形状和 RoPE 配对约束，常见值集中在 64、128 等硬件友好尺寸。

¹¹ 视频画面时间区间：00:45:00–00:45:15。

How many heads, whats the model dim?

Some examples of this hyperparameter

	Num heads	Head dim	Model dim	Ratio
GPT3	96	128	12288	1
T5	128	128	1024	16
T5 v1.1	64	64	4096	1
LaMDA	128	128	8192	2
PaLM	48	258	18432	1.48
LLaMA2	64	128	8192	1
Qwen 3.5 (27B)	24	256	5120	1.2

Most models have ratios around 1 – notable exceptions by some google models.

图 12: 不同模型中的头数、每头维度与模型宽度: 多数模型保持总注意力维度接近隐藏维度。¹²

6.3 宽深比与扩展路径

模型宽度除以层数可视为一种“宽深比”。调查显示, 大量模型落在约 100–200 的量级, 但同参数量下的最佳宽深分配还受训练并行、流水线气泡、单 token 延迟和内存布局影响。更深模型增加顺序依赖和 kernel 次数; 更宽模型则放大单层矩阵并可能提高通信压力。

Aspect ratios

Should my model be deep or wide? *How deep and how wide?*

Most models are surprisingly consistent on this one too!

	Model	d_{model}/n_{layer}
Sweet spot?	BLOOM	205
	T5 v1.1	171
	PaLM (540B)	156
	GPT3/OPT/Mistral/Qwen /OLMo 3	128
	LLaMA / LLaMA2	102
	Gemma 3	87
	Gemma 4	61
	T5 (11B)	33

图 13: 模型宽深比调查: 经验区间集中, 但系统约束会推动不同的扩展路径。¹³

¹²视频画面时间区间: 00:50:45–00:51:00。

扩展模型时要同时做的三张表

一张表记录参数与理论 FLOPs；一张表记录训练吞吐、并行效率和显存；第三张表记录推理的首 token 延迟、逐 token 延迟与 KV 缓存。只沿“参数等量线”扩展，可能得到训练或部署上不可用的形状。

6.4 词表大小是一项系统设计

早期英文单语模型常使用 3 万到 5 万左右词表；现代多语与生产模型常扩大到 10 万至 25 万。大词表能减少多语文本的 token 数和序列长度，却会增大 embedding、输出投影与 softmax 的参数和计算，并改变稀有 token 的训练频率。

What are typical vocabulary sizes?	
Monolingual models – 30-50k vocab	
Model	Token count
Original transformer	37000
GPT	40257
GPT2/3	50257
T5/T5v1.1	32128
LLaMA	32000
Multilingual / production systems 100-250k	
Model	Token count
mT5	250000
PaLM	256000
GPT4	100276
Gemma 4	262144
DeepSeek	100000
Qwen 15B	152064
Yi	64000
Monolingual vocabs don't need to be huge, but multilingual ones do	

图 14: 主流模型词表大小：从英文单语的数万，扩展到多语 / 代码模型的十万量级。¹⁴

词表的完整成本函数

评价词表时，应同时考虑每字符 token 数、序列长度、长尾覆盖、embedding / 输出层参数、softmax 成本、跨语言公平性以及工具 / 代码字符集的兼容性。词表越大并不自动越好。

6.5 本章小结

尺寸比例提供了可靠起点：传统 FFN 约 $4d$ 、门控 FFN 约 $8d/3$ 、总头维度接近模型宽度。最终取值必须结合硬件整除、并行策略、推理延迟和语料分布共同确定。

¹³ 视频画面时间区间：00:51:30–00:51:45。

¹⁴ 视频画面时间区间：00:55:30–00:55:45。

7 正则化与优化动力学：旧概念的新解释

7.1 Dropout 消退，权重衰减保留

许多早期 Transformer 同时使用 dropout 和权重衰减；在数据规模巨大、训练 token 远多于参数的现代语言模型中，dropout 常被完全移除。权重衰减却仍十分普遍，这个反差说明不能只用“小数据过拟合”解释正则化。

Dropout and weight decay in practice			
Model	Dropout*	Weight decay	
Original transformer	0.1	0	
GPT2	0.1	0.1	
T5	0.1	0	
GPT3	0.1	0.1	Many older models used dropout during pretraining
T5 v1.1	0	0	
PaLM	0	(variable)	Newer models (except Qwen) rely only on weight decay
OPT	0.1	0.1	
LLaMA	0	0.1	
Qwen 14B	0.1	0.1	

* Most of the times papers just don't discuss dropout. On open models, this closely matches not doing dropout. This may not be true of closed models.

图 15: 主流模型中的正则化实践：dropout 常为零，权重衰减仍是高频选择。¹⁵

7.2 权重衰减不只是“防过拟合”

以 AdamW 为例，解耦权重衰减可写成：

$$\theta_{t+1} = \theta_t - \eta_t \hat{u}_t - \eta_t \lambda \theta_t.$$

- θ_t ：第 t 步参数； $\hat{u}_t$ ：Adam 预条件后的更新方向；
- η_t ：学习率，因此衰减强度随学习率日程变化；
- λ ：权重衰减系数。

这个更新不仅控制参数范数，也会改变有效步长、尺度平衡和优化轨迹。学习率与权重衰减相乘意味着两者高度耦合：改变 warmup、峰值学习率或衰减日程后，原来的 λ 不一定仍合适。

¹⁵ 视频画面时间区间：01:01:00–01:01:15。

Why weight decay LLMs?

[Andriushchenko et al 2023] has interesting observations about LLM weight decay

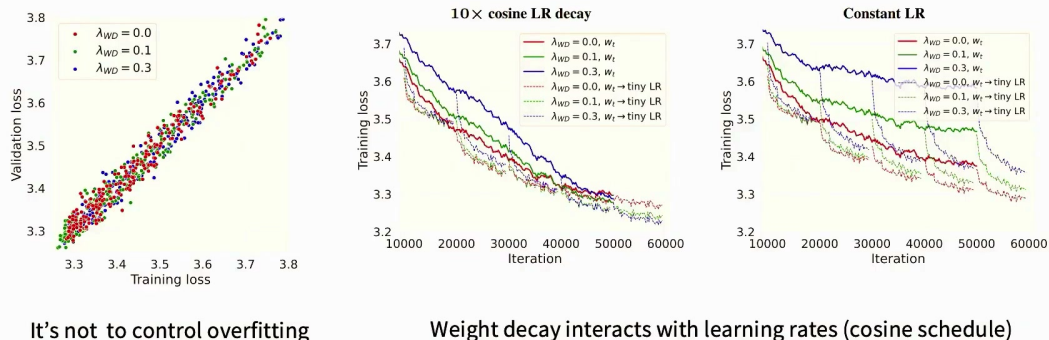


图 16: 权重衰减与学习率 / 训练动力学的耦合：其作用不能简化为验证集正则化。¹⁶

迁移训练配方的常见错误

从另一规模复制权重衰减时，若同时改变 batch size、学习率、训练长度或参数化方式，就已经改变了每个参数累计受到的衰减。应比较整段训练中的积分效应，而不是孤立比较一个 λ 数字。

7.3 本章小结

现代大规模训练减弱了 dropout 的必要性，却没有消除权重衰减。后者更像优化动力学的一部分：它与学习率、参数尺度和训练时长共同决定轨迹，必须成组调节。

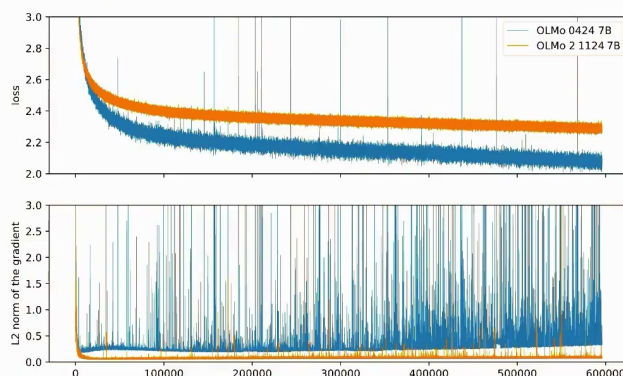
8 训练稳定性：控制 Softmax 的危险尺度

模型规模越大、训练周期越长，一次不可恢复的数值爆炸就越昂贵。现代架构研究因此不只追求平均验证损失，还要降低训练尖峰、NaN 和发散的尾部风险。讲者指出，许多稳定性干预都指向同一处：Softmax 中的指数与归一化除法。

¹⁶ 视频画面时间区间：01:01:30–01:01:45。

Stability tricks

Recently, lots of attention on *stable training*



Don't train models that look like the blue curve!

图 17: 训练稳定性曲线: 短暂损失尖峰可能自行恢复, 也可能摧毁一次高成本训练。¹⁷

8.1 语言模型中有两处关键 Softmax

第一处是输出词表 Softmax, 用于把 logits 转为下一个 token 的概率; 第二处是注意力 Softmax, 用于把 $QK^\top$ 转为加权系数。数值实现通常会先减去最大 logit 避免上溢, 但若 logit 尺度持续增大, 仍会造成极端饱和、低精度误差、梯度异常和训练脆弱性。

稳定性设计的共同模式

不要等到最终损失变成 NaN 才诊断。应持续监控输出 logits、注意力 logits、隐藏状态 RMS、梯度范数、更新 / 参数范数比以及每层异常值。稳定性组件的价值在于提前限制危险内部量。

8.2 z-loss: 约束输出归一化常数

设输出 logits 为 $z_1, \dots, z_V$, Softmax 的对数配分函数为 $\log Z = \log \sum_j e^{z_j}$ 。z-loss 在交叉熵外增加一个小惩罚:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \alpha \left(\log \sum_{j=1}^V e^{z_j} \right)^2.$$

- V : 词表大小; z_j : 第 j 个词的输出 logit;
- $\mathcal{L}_{\text{CE}}$: 标准交叉熵损失;
- α : 通常很小的 z-loss 权重, 用于抑制整体 logit 尺度漂移。

¹⁷ 视频画面时间区间: 01:05:15–01:05:30。

Softmax 对所有 logits 同加常数不敏感，交叉熵因此没有动力控制这个共同偏移；z-loss 显式给该“自由方向”加上软约束。它主要是数值保险，而不是要改变正确类别与错误类别的相对排序。

Output softmax stability – the ‘z-loss’

Recall the softmax calculation

$$\begin{aligned}\log(P(x)) &= \log\left(\frac{e^{U_r(x)}}{Z(x)}\right) \\ &= U_r(x) - \log(Z(x)) \\ Z(x) &= \sum_{r'=1}^{|V|} e^{U_{r'}(x)}\end{aligned}$$

$$\begin{aligned}L &= \sum_i [\log(P(x_i)) - \alpha(\log(Z(x_i)) - 0)^2] \\ &= \sum_i [\log(P(x_i)) - \alpha \log^2(Z(x_i))]\end{aligned}$$

[From Devlin 2014]

This is useful for stability! PaLM used this ‘z loss’ trick.

We additionally use an auxiliary loss of $z_loss = 10^{-4} \cdot \log^2 Z$ to encourage the softmax normalizer $\log(Z)$ to be close to 0, which we found increases the stability of training.

Other examples: Baichuan 2 (2023), DCLM (2024), OLMo 2 (2025), OLMo 3 (2025)

图 18: z-loss: 惩罚输出 Softmax 的对数配分函数，限制无意义的整体 logit 漂移。¹⁸

8.3 QK-Norm: 在注意力 Softmax 之前控幅

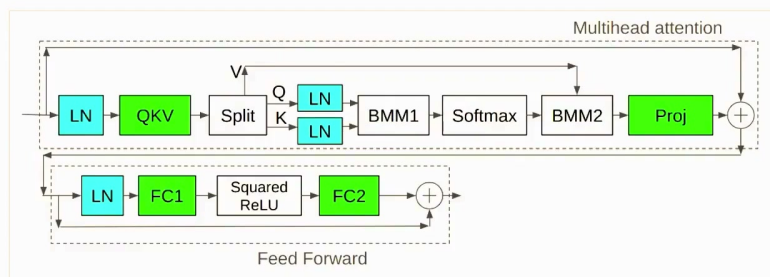
注意力 logits 通常为 $QK^\top/\sqrt{d_h}$ 。即使有 $1/\sqrt{d_h}$ 缩放，训练中的查询和键范数仍可能增长。QK-Norm 在内积之前分别归一化 Q 与 K ：

$$A = \text{softmax}\left(\frac{\text{Norm}(Q) \text{Norm}(K)^\top}{\sqrt{d_h}}\right) V.$$

- Q, K, V : 查询、键和值矩阵；
- d_h : 单个注意力头的维度；
- Norm: 常为沿 head dimension 的 RMSNorm 或 LayerNorm；
- A : 注意力加权后的输出。

¹⁸视频画面时间区间：01:07:15–01:07:30。

Attention softmax stability – the ‘QK norm’



The query and keys are Layer (RMS) normed before going into the softmax operation.

Other examples: DCLM, OLMo2, Gemma 2, Qwen3, OLMo 3, Gemma 4

Originally from vision and multimodal models [Dehgani 2023, Idefcs, Chameleon]

🔍 🔍 🔍 🔍 🔍 🔍

图 19: QK-Norm: 在查询与键形成内积前归一化, 直接限制注意力 logits 的尺度。¹⁹

另一类方法是 soft-capping, 例如把 logit z 映射为 $c \tanh(z/c)$ 。它给 logit 设置硬上界, 但也会压缩大间隔区域, 可能牺牲质量。 z -loss、QK-Norm 和 soft-cap 的共同目的都是控幅, 干预位置和质量代价却不同, 不能互相简单替代。

8.4 稳定性实验该怎样设计

平均损失不足以评价稳定性。至少应在多个随机种子上报告: 尖峰次数与幅度、是否自动恢复、最大 logit / 激活 / 梯度、恢复后最终质量, 以及干预带来的吞吐成本。对于超大训练, 可先在更小规模上人为提高学习率或降低精度, 构造“压力测试”, 但仍需确认干预在目标规模不过度约束表达能力。

8.5 本章小结

输出 Softmax 与注意力 Softmax 是两处核心危险区。 z -loss 约束输出配分函数, QK-Norm 约束查询 / 键尺度, soft-cap 则直接截住极端 logits。它们是面向昂贵训练的风险控制组件, 应以尾部失败概率而非只看平均损失来评估。

9 推理系统: Prefill、Decode 与 KV 缓存

训练完成后, 架构约束会从“如何高效并行训练”转向“如何低延迟服务”。自回归推理分为两个性质截然不同的阶段: Prefill 一次处理整段提示词; Decode 每步只生成一个新 token, 并依赖此前所有缓存状态。

¹⁹ 视频画面时间区间: 01:10:15–01:10:30。

9.1 Prefill: 矩阵大、复用高、较偏计算受限

对长度为 L 的提示词，注意力可并行生成全部查询、键和值，再进行大矩阵乘法。每次从显存读入权重后可服务许多 token，数据复用高；足够大的 batch 和序列常能接近 GPU 计算峰值。标准注意力的主要计算随 L^2 增长，因此长提示仍可能昂贵。

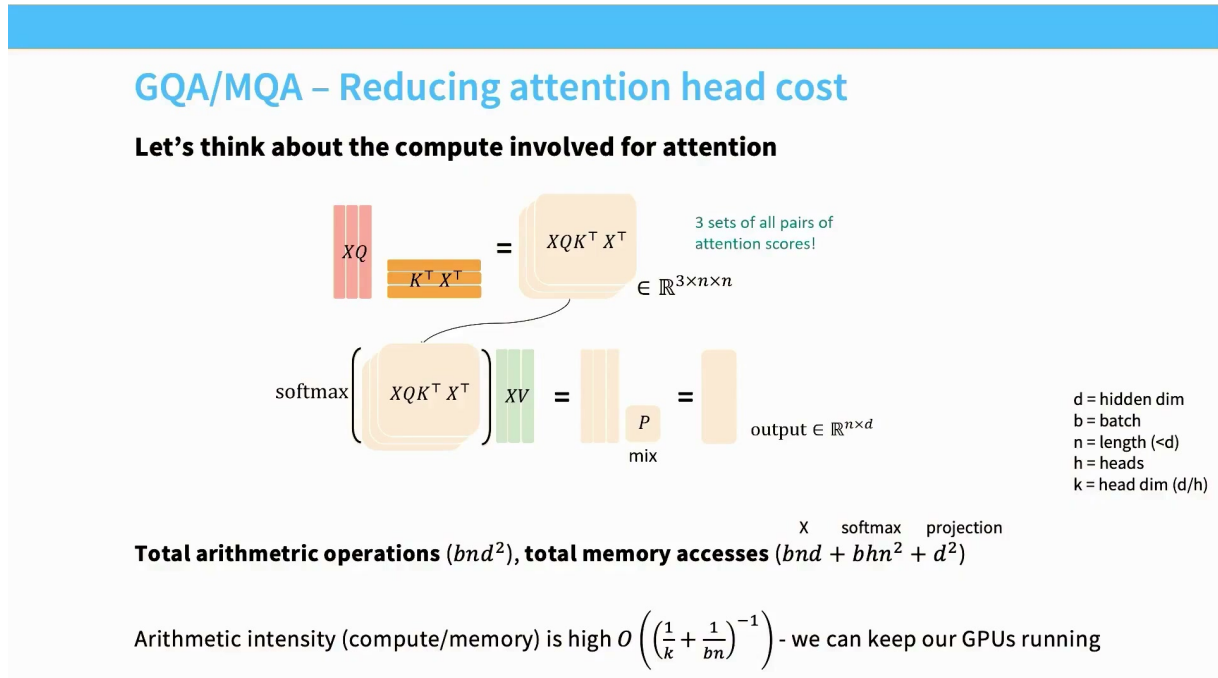


图 20: Prefill 阶段的注意力：大规模并行矩阵运算带来较高算术强度。²⁰

算术强度可粗略理解为单位显存传输产生的浮点运算：

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{Bytes moved}}.$$

- 分子：矩阵乘法等完成的浮点运算量；
- 分母：在显存与计算单元之间搬运的字节数；
- 比值高时更可能受计算吞吐限制，比值低时更可能受带宽限制。

9.2 Decode: 顺序依赖强、算术强度低

Decode 的第 t 步只有一个新查询，但必须读取过去 t 个 token 的键和值。KV 缓存避免了重算历史状态，却把瓶颈转成每步读取大量缓存。生成下一 token 后才能开始下一步，因此时间维无法像 Prefill 一样并行。

²⁰ 视频画面时间区间：01:15:15–01:15:30。

GQA/MQA – Reducing attention head cost

What's the incremental arithmetic intensity?

Total arithmetic operations (bnd^2), **total memory accesses** ($bn^2d + nd^2$)^{K, V projection}

Arithmetic intensity is not good $O\left(\left(\frac{n}{d} + \frac{1}{b}\right)^{-1}\right)$ - need large batches + short seq length (n) or big model dimensions (d)

Is there some way around this? The n/d term is difficult to reduce.



图 21：增量解码：每步仅产生一个查询，却要读取全部历史 KV，常表现为显存带宽受限。²¹

标准多头注意力的 KV 缓存元素量近似与下式成正比：

$$N_{KV} \propto 2BLn_{\text{layers}}n_{\text{kv}}d_h.$$

- B ：服务 batch size； L ：已缓存上下文长度；
- n_{layers} ：层数； n_{kv} ：每层 KV 头数；
- d_h ：每个 KV 头维度；系数 2 对应 K 与 V 两份缓存。

缓存字节数还需乘以数据类型字节宽度。这个关系解释了为什么减少 KV 头数、压缩缓存精度和限制局部窗口会直接提升服务容量。

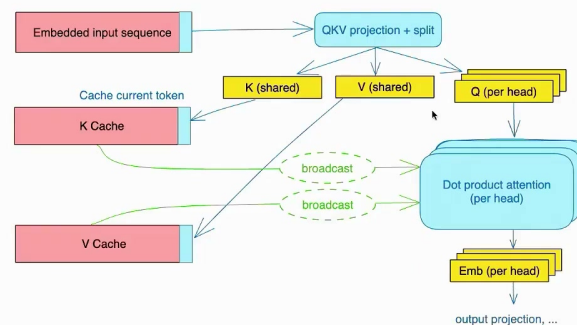
9.3 MQA：所有查询头共享一组 K/V

Multi-Query Attention (MQA) 保留多个查询头，却只使用一组键和值。与标准 MHA 相比，KV 缓存可近似按查询头数倍缩小，Decode 时的数据读取也随之下降。

²¹ 视频画面时间区间：01:18:15–01:18:30。

MQA – just have fewer key dimensions.

Key idea – have multiple queries, but just one dimension for keys and values



We have much fewer items to move in and out of memory (KV Cache)

Total memory access $(bnd + bn^2k + nd^2)$, **Arithmetic intensity** $O\left(\left(\frac{1}{d} + \frac{n}{dh} + \frac{1}{b}\right)^{-1}\right)$



[figure from <https://blog.fireworks.ai/multi-query-attention-is-all-you-need-db072e758055>]

图 22: MQA: 各查询头仍独立, 但共享同一组键和值, 大幅压缩 KV 缓存。²²

MQA 的代价是键 / 值表示的多样性下降, 可能造成质量损失。对于极端受内存带宽限制的在线服务, 这种交换仍可能值得; 但不能只根据缓存缩小倍数判断总体收益。

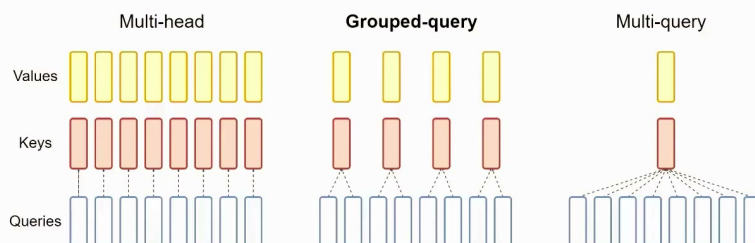
9.4 GQA: 系统效率与表达能力的折中

Grouped-Query Attention (GQA) 把查询头划分为若干组, 每组共享一对 K/V。它在 MHA 与 MQA 之间连续调节 n_{kv} : KV 头越少, 缓存越小; KV 头越多, 表示越丰富。很多现代模型采用 GQA, 因为它常能取得显著系统收益而质量损失很小。

²² 视频画面时间区间: 01:19:45–01:20:00。

Additional extensions – GQA

Don't go all the way to one dimension of KV – have fewer dims



Simple knob to control expressiveness (key-query ratio) and inference efficiency

More recently – MLA (multihead latent attention) from deepseek v2

图 23: MHA、GQA 与 MQA: 通过改变每组查询头共享的 KV 数量调节质量 / 内存折中。²³

部署指标必须分阶段报告

Prefill 应重点报告首 token 延迟、长提示吞吐和峰值注意力内存；Decode 应报告逐 token 延迟、并发容量、KV 缓存字节 / token 和带宽利用率。把两阶段平均成一个 tokens/s，容易掩盖真正瓶颈。

9.5 本章小结

Prefill 有大矩阵和高数据复用，Decode 则有严格顺序依赖并频繁读取 KV 缓存。MQA 和 GQA 不是纯粹的建模花样，而是直接面向 Decode 带宽与服务容量的架构设计。

10 长上下文：稀疏、滑动窗口与混合注意力

全局注意力允许任意 token 访问全部过去信息，但计算和缓存随上下文快速增长。长上下文设计的关键问题是：是否每一层、每一个 token 都需要全局通信？如果局部层能完成大部分短程处理，只需周期性注入全局层，就可能显著降低成本。

10.1 从稀疏模式到滑动窗口

稀疏注意力只计算预先选择的连接。最常见的滑动窗口让位置 i 只访问最近 w 个历史位置，把每层注意力计算从 $O(L^2)$ 降到近似 $O(Lw)$ ，同时把该层 KV 缓存限制在窗口内。

²³ 视频画面时间区间：01:20:45–01:21:00。

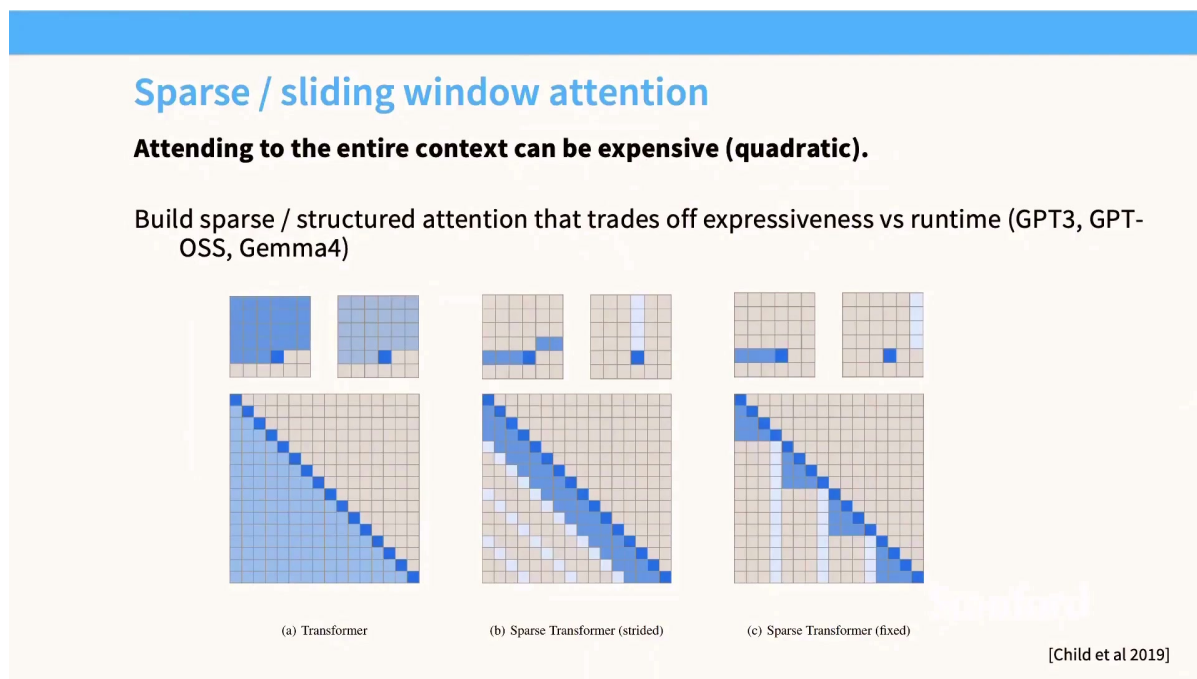


图 24: 全局、稀疏与滑动窗口注意力模式：减少连接数量即可降低长序列成本。²⁴

局部窗口的“层间传播”不等于直接检索

堆叠局部层会扩大理论感受野，但远处信息必须经过多层逐步传递，路径更长且可能失真。需要精确访问远端 token 的任务，通常仍需周期性全局层、检索机制或其他显式长程通道。

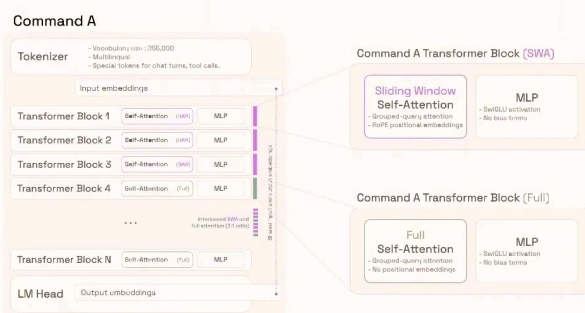
10.2 交错全局层与局部层

一种实用方案是让大多数层使用便宜的局部注意力，间隔若干层插入全局注意力。Command A 等模型展示了这种思路：局部层以 RoPE 和滑动窗口处理邻域模式，周期性全局层重新建立跨序列通信。这样，每个 token 先被局部计算充分加工，再通过全局层汇聚远程信息。

²⁴视频画面时间区间：01:25:15–01:25:30。

Current standard trick – interleave ‘full’ and ‘LR’ attention

From Cohere Command A – Every 4th layer is a full attention



Long-range info via NoPE, short-range info via RoPE + SWA.

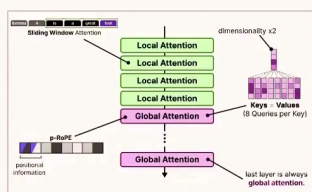
Other models – LLaMA 4, Gemma 3, Gemma 4, OLMo 3 does SWA+Full RoPE.

图 25: 全局 / 局部注意力交错: 用少量全局层维持远程通路, 用多数局部层控制成本。²⁵

10.3 混合架构仍在快速演化

近期模型把“便宜层”扩展到多种形式: 局部滑窗、不同频率的全局注意力, 甚至状态空间层与注意力层混合。它们共享同一原则: 高成本全局通信不必在每层发生。区别在于信息如何跨越局部窗口、位置编码施加在哪里、缓存如何管理, 以及硬件上是否有成熟内核。

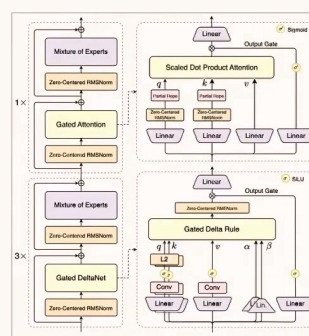
Other recent examples of interleaved attention



Gemma 4

Gradient clipping	1.0
Z-loss weight	10^{-5}
Weight decay on embeddings	No
Sliding window attention	3/4 of layers: 4,096 tokens
RoPE scaling	YaRN on full attn. layers
RoPE θ	$5 \cdot 10^6$
Layer norm applied to	Outputs

Olmo 3



Qwen 3.5 / Qwen 3 Next

<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-gemma-4>

图 26: 近期长上下文混合架构: Gemma、OLMo、Qwen 等采用不同的局部 / 全局或状态空间组合。²⁶

²⁵ 视频画面时间区间: 01:26:30–01:26:45。

长上下文评估不能只看最大长度

“支持 1M token” 是接口上限，不是有效利用能力。应分别测试短程保持、远程检索、多跳组合、位置鲁棒性、不同上下文长度下的质量退化，以及 Prefill / Decode 的真实成本。

10.4 本章小结

滑动窗口把每层成本从全局二次连接降为局部带状连接；交错架构用少量全局层补回远程通信。长上下文仍是设计变化最快的部分，必须联合评估信息可达性、训练分布、内核效率和缓存成本。

11 总结与延伸

11.1 讲者总结：共识、例外与变化最快的边界

课程最后回到架构调查表。今天的现代语言模型存在清晰共识：Pre-Norm 或其他经过稳定性验证的残差方案、RMSNorm、RoPE、门控 FFN，以及为部署优化的 GQA。与此同时，宽深比、词表、正则化和长上下文模式仍存在大量成功例外。

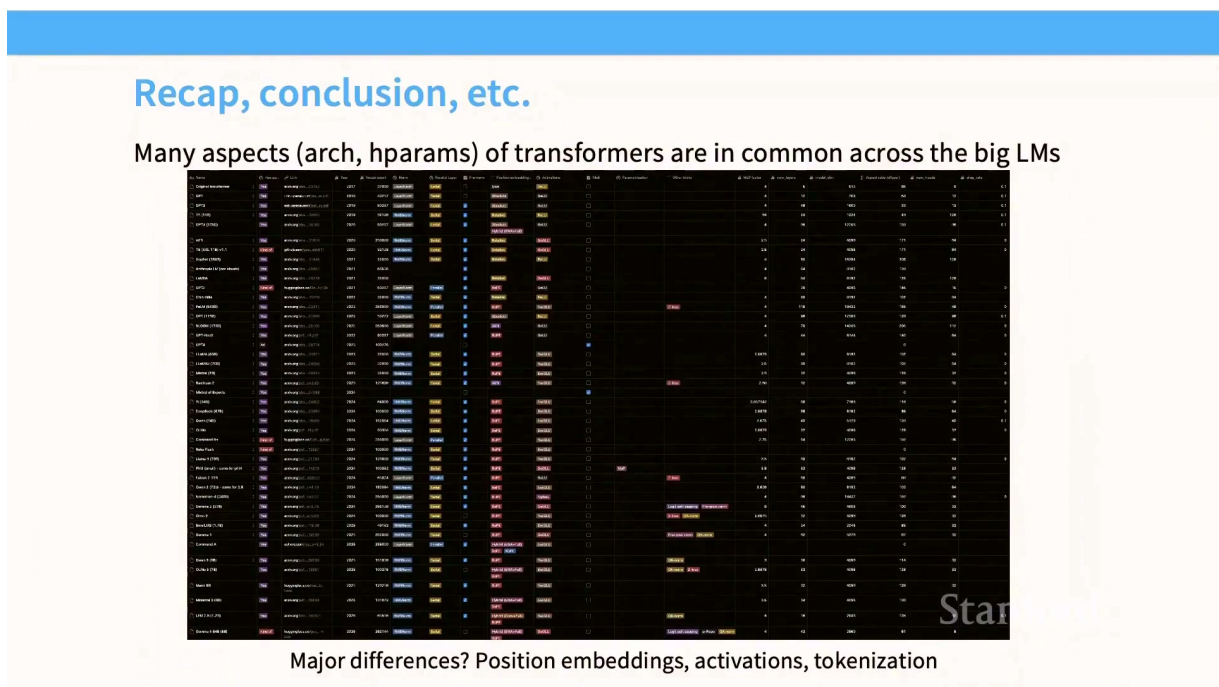


图 27：课程回顾：主流架构已出现稳定默认值，但位置与上下文处理仍快速变化。²⁷

11.2 一张可执行的设计清单

1. **先定义约束。**写清训练预算、目标硬件、序列长度、并发与质量门槛；没有约束就不存在“最佳架构”。

²⁶ 视频画面时间区间：01:28:00–01:28:15。

²⁷ 视频画面时间区间：01:28:45–01:29:03。

2. **建立成熟基线。**从 Pre-Norm / RMSNorm、RoPE、SwiGLU、GQA 等可靠默认值开始，先复现损失和吞吐。
3. **区分建模与系统指标。**同时测验证损失、FLOPs、墙钟时间、显存、通信、首 token 延迟和逐 token 延迟。
4. **把稳定性当一等指标。**监控两处 Softmax 的 logits、隐藏状态与梯度；在多随机种子下报告尖峰与发散率。
5. **逐项偏离并做等预算消融。**组件变化要控制数据、参数量、训练 token 和优化器，避免把品牌相关性误写成果。
6. **单独验证长上下文。**最大输入长度、远程信息利用率和服务成本是三件不同的事。

本讲最重要的综合判断

现代架构优化的核心，不只是增加表达能力，而是把有限的参数、计算、显存带宽与数值风险分配到最有价值的位置。很多“新组件”实际上是在重新安排数据搬运、缓存或危险内部尺度。

11.3 进一步推导：从组件思维转向约束思维

把全课放在一起，可以得到三条更一般的推论。

第一，**架构与系统不可分割。**RMSNorm、并行块、GQA、滑动窗口之所以重要，不只因为数学形式，也因为它们改变内存访问、并行度和缓存体积。理论 FLOPs 相同的两个模型，真实成本可能完全不同。

第二，**稳定性是规模化能力的一部分。**在小实验中偶发一次尖峰可能无关紧要；在持续数月的大训练中，尾部失败概率会直接决定项目经济性。z-loss 与 QK-Norm 代表一种成熟工程观：提前限制危险变量，而不是期待优化器自行修复。

第三，**上下文是下一轮架构分化的中心。**基础块已趋于同质化，模型间更显著的差别转向位置编码、局部 / 全局通信比例、KV 缓存和混合序列模块。评价这些设计时，应从“标称窗口有多长”转向“每单位成本能可靠利用多远的信息”。

11.4 建议练习

1. 在同一小型语言模型上比较 LayerNorm 与 RMSNorm：同时测损失、tokens/s 和归一化 kernel 时间。
2. 实现 MHA、GQA 与 MQA，在固定参数预算下测 KV 字节 / token、Decode 延迟及困惑度变化。
3. 记录训练过程中的输出 log-sum-exp、注意力 logits 最大值和梯度范数，再比较加入 z-loss 或 QK-Norm 后的尖峰分布。
4. 构造需要跨越局部窗口检索的信息，比较全局注意力、纯滑窗和交错全局 / 局部层的正确率与吞吐。

11.5 拓展阅读

- Stanford CS336 Spring 2026 课程主页: <https://cs336.stanford.edu/>
- Lecture 3 官方课件: https://github.com/stanford-cs336/lectures/blob/main/lecture_03.pdf
- RoFormer (RoPE 原始论文): <https://arxiv.org/abs/2104.09864>
- GLU Variants Improve Transformer: <https://arxiv.org/abs/2002.05202>
- GQA: Training Generalized Multi-Query Transformer Models: <https://arxiv.org/abs/2305.13245>
- FlashAttention (从 I/O 复杂度理解注意力): <https://arxiv.org/abs/2205.14135>

11.6 本章小结

可靠的现代基线让我们少走弯路，例外则提醒我们继续测量。模型设计的成熟路径是：明确约束、选择默认值、建立端到端指标、控制变量做消融，并把稳定性和部署成本放进与损失同等重要的验收表。